Statute Co-Citation Graph Prefixes for Out-of-Time Judgment Outcome Prediction at the Supreme Court of India

Akul Ranjan and Vansh Agarwal

Manuscript, October 2026

Abstract. Predicting how an appeal will be disposed of is a long-standing task in legal artificial intelligence. Reported results are hard to compare, however: evaluations often leak outcome cues into the input, mix time periods across training and test data, and seldom isolate whether structured legal knowledge adds anything to what a large language model reads. We study four-way outcome prediction (allowed, dismissed, disposed of, partly allowed) for judgments of the Supreme Court of India under a leakage-controlled, out-of-time protocol. From two public sources we assemble 2,470 training judgments decided between 2013 and 2016. Their text-derived labels are checked against the court's official disposal metadata. We hold out 608 test judgments, 558 of them decided in 2017, after every training case. Models read only a masked opening of the judgment body. We propose a *statute co-citation graph prefix*. The provisions cited in the visible text seed a time-safe and split-safe subgraph, in which statutes are linked when earlier training judgments cite them together. A relational graph convolutional network compresses this subgraph into eight soft tokens that condition a 4-bit Qwen3-8B model adapted with QLoRA. The prefix adds 6,340,097 parameters and less than one percent of training time. Every model is frozen and hashed before a single test evaluation. On the full test set, both fine-tuned models more than double the macro-F1 of a majority-class baseline (0.159). They also improve on the same model used without training by 0.090 (text) and 0.096 (graph) macro-F1, with bootstrap intervals that exclude zero. On the 2017 out-of-time test set, the graph-conditioned model attains the highest macro-F1 of all systems (0.379 against 0.365 for the text-only model). The gain comes mainly from markedly better recognition of dismissed appeals (F1 0.395 against 0.272). The text-only model achieves higher accuracy and leads on the smaller 2016 test set. On the 2017 set, the macro-F1 and accuracy differences between the two fine-tuned models lie within paired bootstrap confidence intervals. Across development stages, a balanced training recipe, a larger training set and a statute-only prefix together narrowed the validation macro-F1 gap between graph-conditioned and text-only models. At the first stage the graph model scored 0.244 against 0.441; at the final stage it scored 0.385 against 0.405. Diagnostic probes identify task framing and training-set size as the strongest remaining levers. We release the data-construction code, configurations, hashes and analysis scripts to support replication.

Keywords: legal judgment prediction; Supreme Court of India; knowledge graphs; graph neural networks; large language models; QLoRA; soft prompts; temporal evaluation; data leakage.

1 Introduction

Legal judgment prediction asks a model to anticipate the outcome of a case from its text. The task has been studied for the European Court of Human Rights [1–3], the Supreme Court of the United States [4], Chinese criminal courts [5, 6] and the Swiss Federal Supreme Court [7]. Its appeal is practical as well as scientific. Systems that read case files can support triage, legal research and empirical studies of courts. The task also probes whether models capture something of legal reasoning rather than

surface vocabulary [8, 9]. India is a natural setting for this work. Its apex court publishes a large volume of English-language judgments, and the community has produced dedicated corpora, benchmarks and pre-trained models [10–12].

Large language models (LLMs) now read legal text fluently, and parameter-efficient fine-tuning makes it practical to adapt 7–8 billion-parameter models on a single commodity GPU [13, 14]. Yet an LLM still receives a judgment as a flat sequence of tokens. Legal reasoning is relational. A judgment applies provisions of statutes, sits in a web of precedent, and is bound to the law as it stood on the decision date. Earlier work has used such structure to good effect: dependencies among subtasks [5], graphs that separate confusable law articles [15], heterogeneous statute–document networks [16, 17] and relational learning over cases [18]. In open-domain question answering, graph encoders that feed a language model, either through joint reasoning [19, 20] or through soft graph tokens [21–23], have shown that a compact graph signal can be fused with a frozen or lightly tuned LLM. Whether such a signal helps legal outcome prediction, under realistic constraints, is an open question.

Answering it requires care with evaluation. Judgments state their own outcome: the operative paragraph, and often a headnote, announces that the appeal is allowed or dismissed. Models trained on unmasked text can therefore learn to read the answer rather than predict it [24]. Random splits overstate performance when the data drift over time [25, 26]. Test sets of a few dozen cases cannot separate competing systems, and evaluating repeatedly on the test set inflates results through selection [27, 28]. A graph built from the corpus is a further channel of leakage. If test judgments, or judgments decided after the query, contribute edges, the graph can carry information that a real user would not have.

This paper studies graph-conditioned outcome prediction for the Supreme Court of India under a protocol designed to close these channels. We ask three research questions:

1. **RQ1.** How well does a QLoRA-adapted 8B LLM predict four-way outcomes from a masked opening of the judgment, when it is tested on judgments decided after all of its training data?
2. **RQ2.** Does a compact statute graph prefix, built only from earlier training judgments, change what the model predicts, and for which kinds of cases?
3. **RQ3.** What limits performance in this setting, and which levers promise the largest gains?

Our contributions are as follows.

1. **A leakage-controlled, out-of-time benchmark.**
   - **Data.** We combine the SC-2016 judgment collection [29] with judgments from a public open-data archive of the Supreme Court of India [30]. This yields 2,470 labelled training judgments (2013–2016), a 46-judgment validation set and a 608-judgment test set. 558 of the test judgments were decided in 2017, after every training case.
   - **Labels.** Text-derived labels are validated against the court's official disposal metadata, which agrees on 97.7% of overlapping cases. A judgment is retained only when the two agree.
   - **Pinning.** Every source file is pinned by SHA-256.
2. **A statute co-citation graph prefix for quantised LLMs.**
   - **Subgraph.** Statute provisions cited in the visible text seed a subgraph built from training-split, time-eligible edges only. Provisions are linked when earlier judgments cite them together, a legal analogue of bibliographic co-citation [31].

- **Encoder.** A relational graph convolutional network [32] with query-conditioned pooling compresses the subgraph into eight soft tokens that are prepended to the prompt [21, 33].
- **Contents.** The prefix contains no case nodes, titles or outcomes.

3. **A strict evaluation protocol.**
   - **Freezing.** Models are trained, frozen and hashed in one environment. They are scored on the test set exactly once, in a separate environment that verifies the hashes.
   - **Reporting.** Results are reported per source, with 95% bootstrap intervals, paired bootstrap differences and exact McNemar tests [34, 35].
   - **Ablation.** The graph model is also scored with its graph tokens removed.
4. **Empirical findings.**
   - **Fine-tuned vs majority.** Both fine-tuned models more than double the majority-class macro-F1 on the full and the 2017 test sets.
   - **2017 test.** On the out-of-time 2017 test set, the graph-conditioned model has the highest macro-F1 of all systems (+0.014 over the text-only model). Its dismissed-class F1 rises from 0.272 to 0.395. The text-only model retains higher accuracy.
   - **Development trajectory.** We trace the stages, from a co-citing-case prefix to a statute-only prefix, over which the validation gap between graph and text models closed.
5. **Diagnostics and a research roadmap.** Probes on held-out training years quantify how much the input view, the training-set size and the task framing matter. They point to concrete next steps.

Section 2 reviews related work. Section 3 describes the data and benchmark construction, Section 4 the method, and Section 5 the experimental setup. Section 6 reports results and Section 7 discusses them. Section 8 sets out directions for further research, and Section 9 concludes.

2 Related Work

2.1 Legal judgment prediction

Early work. Early work framed outcome prediction as text classification over hand-engineered or bag-of-words features. Aletras et al. predicted violations of the European Convention on Human Rights from n-gram and topic features of case descriptions [1]. Katz et al. predicted the behaviour of the US Supreme Court from case and justice metadata with random forests, over two centuries of decisions [4]. Medvedeva et al. examined how such models behave when trained on earlier decisions and tested on later ones [2].

Neural models. Neural approaches followed. Chalkidis et al. introduced an English ECHR dataset and hierarchical attention and BERT baselines [3]. The large Chinese CAIL2018 corpus [6] spurred models that exploit the structure of the label space, such as topological dependencies among the charge, law-article and penalty subtasks [5]. Others distinguish confusable law articles with graph distillation [15], model circumstances [36] or extract legal events under constraints [37]. Relational learning across cases has also been explored [18].

Benchmarks and surveys. Multilingual and multi-task benchmarks now cover Swiss, European and US data [7, 38, 39]. Surveys summarise the field [8, 9].

Critiques. A growing body of work questions what such models actually predict. Medvedeva and McBride argue that many "prediction" studies classify outcomes from texts written after the decision, and so describe rather than forecast [24]. Rationale-extraction work on ECHR cases shows that the paragraphs a model relies on can diverge from those legal experts consider decisive [40]. These critiques motivate our masked-prefix input view and out-of-time test.

2.2 Legal NLP for Indian courts

Corpora and models. The Indian Legal Documents Corpus (ILDC) introduced court judgment prediction and explanation for the Supreme Court of India. It pairs binary accept/reject labels with expert explanations, and models read the judgment text with the final decision removed [10]. IL-TUR assembles a broader benchmark of Indian legal understanding and reasoning tasks [11]. InLegalBERT and related models pre-train on Indian legal text [12]. Rhetorical-role corpora segment judgments into facts, arguments, analysis and ruling [41].

Statute and similarity graphs. LeSICiN identifies relevant statutes with a heterogeneous graph of documents and statute sections [16]. Hier-SPCNet uses the statute hierarchy to compute case similarity [17].

LLMs. Recent studies evaluate LLMs on Indian statute and judgment prediction [42] and on prediction with explanation [43].

How our setting differs.

- **Labels.** We keep four outcome classes, including the procedurally important "disposed of" category. Labels are validated against the court's own metadata.
- **Inputs.** We restrict inputs to the opening of the judgment body.
- **Test set.** We test on a later year.

2.3 Legal language models and LLMs

Domain models. Domain-specific pre-training improves legal understanding: LEGAL-BERT [44] for English, InLegalBERT [12] for Indian law. It is supported by large open legal corpora such as the Pile of Law [45]. Domain-adaptive pre-training is a general recipe for such gains [46]. SaulLM-7B continues pre-training a 7B decoder on legal text [47].

Evaluation and risks. LegalBench measures legal reasoning across many task types [48]. Systematic studies of LLMs for judgment prediction find strong but uneven zero-shot and few-shot performance [49]. Precedent-enhanced methods combine LLMs with retrieved similar cases [50]. LLMs also hallucinate legal facts at worrying rates [51], which favours designs that ground predictions in verifiable structure.

2.4 Graphs and networks in law

Network analysis. Citation networks have long been used to measure the importance of precedents [52]. Bibliometric co-citation, in which two documents are related when a third cites both, gives a robust and interpretable similarity signal [31].

Legal knowledge graphs. Knowledge graphs offer a general framework for typed, provenance-backed legal facts [53]. In the Indian context, statute–document graphs support statute identification and similarity search [16, 17].

Our graph. We follow this tradition with a deliberately conservative graph. Provisions are linked only through training-split judgments decided before the query. The prefix contains provisions alone, so no case outcome or party name can reach the model through the graph.

2.5 Fusing graphs with language models

Graph neural networks. GNNs learn node representations by message passing [54]. Relational GCNs extend this to typed edges with relation-specific weights [32].

Fusion approaches. Several lines of work fuse GNNs with language models:

- **Joint reasoning.** QA-GNN and GreaseLM let text and knowledge-graph representations interact in question answering [19, 20].
- **Soft graph tokens.** For decoder LLMs, GraphToken learns to encode a graph into soft tokens consumed by a frozen LLM [21]. Graph Neural Prompting injects a graph neural prompt into an LLM [22].
- **Graph retrieval.** G-Retriever retrieves a subgraph with a prize-collecting Steiner tree (PCST) formulation [55], encodes it with a GNN and feeds it to an LLM together with a textualised graph [23]. Retrieval-augmented generation [56] and its graph variants [57] retrieve structure at inference time.
- **Survey.** A recent roadmap surveys the unification of LLMs and knowledge graphs [58].

Our approach. Our encoder follows the soft-token line, and our first selector followed G-Retriever's PCST formulation. Our final design replaces PCST with a statute co-citation selector, which we found better suited to the leakage constraints of outcome prediction (Section 6.5).

2.6 Parameter-efficient adaptation

Adapters. Low-rank adaptation (LoRA) trains small rank-decomposition matrices alongside frozen weights [13]. QLoRA backpropagates through a 4-bit NormalFloat-quantised base model, which brings 8B-parameter fine-tuning within reach of a 16 GB GPU [14].

Soft prompts. Prefix-tuning showed that continuous vectors prepended to the input can steer a frozen model [33]. Our graph tokens are a data-dependent prefix of this kind, computed from each case's subgraph.

Base model. Qwen3 is a recent open-weight model family with strong multilingual and reasoning performance [59].

2.7 Evaluation methodology

Leakage and splits. Leakage from training into test data is a pervasive source of over-optimism in applied machine learning [27]. Random splits can overstate generalisation compared with splits that respect time [25]. Language models degrade on text from periods after their training data [26].

Statistics. For small test sets, paired resampling tests are recommended over unpaired comparisons [28, 34]. McNemar's test compares two classifiers on the same cases [35]. Under class imbalance, macro-averaged F1 and class re-weighting are standard [60].

2.8 Research gap

Existing Indian judgment-prediction studies mostly evaluate on the judgment text up to the decision and on in-period splits. Graph–LLM fusion has been demonstrated mainly on open-domain question answering. To our knowledge, no prior study combines all of the following:

- a strictly time-safe and split-safe legal graph prefix for a quantised decoder LLM;
- four-way outcomes validated against official metadata;
- an out-of-time test year;
- a single-shot, hash-verified test evaluation with paired statistics.

This paper provides such a study and reports what the graph prefix changes and where.

3 Data and Benchmark Construction

3.1 Sources

Judgments. We use two public collections of judgments of the Supreme Court of India.

- **SC-2016** [29] contains judgments reported in 2016, as OCR-derived Markdown produced with a PaddleOCR pipeline [61]. Its documents are assigned to training, validation and test splits by a deterministic hash: the first bytes of each document's SHA-256 digest, modulo ten.
- **The open-data archive** [30] publishes per-year archives of judgment PDFs and structured metadata under a CC BY 4.0 licence. The metadata includes the decision date, case number, bench, the reported (SCR) and neutral citations, and an official *disposal nature*. The archive is updated periodically, so every archive we used is pinned by SHA-256 in a source registry.

Statutes. The statute layer covers six enactments with 575 + 507 + 184 + 463 + 358 + 532 provision nodes:

- the Indian Penal Code (IPC), the Code of Criminal Procedure (CrPC), the Indian Evidence Act (IEA) and the Constitution of India, taken from commit-pinned public JSON sources [62];
- the Bharatiya Nyaya Sanhita (BNS) and the Bharatiya Nagarik Suraksha Sanhita (BNSS) of 2023.

Each provision carries its heading and text, which we use as node features (Section 4.5).

3.2 Text extraction for the archive judgments

Extraction and cleaning. Archive judgments are distributed as PDFs with an embedded text layer, which we extract with PyMuPDF. Cleaning removes:

- the margin letters (A–H) of the Supreme Court Reports layout;
- page numbers and running "Supreme Court Reports" headers;
- any header line repeated at the top of three or more pages.

It then removes end-of-line hyphenation and joins wrapped lines into paragraphs. Extraction ran in a CPU-only cloud session, and its output is a deterministic, hash-pinned record file.

Quality. On the 357 SC-2016 training judgments that also appear in the archive, about 98% of words in the cleaned text layer are clean, compared with 99.5% for the OCR text. The text layer recovers about 88% of the statute citations found in the OCR text of the same judgments. In our checks, the text

layer and the OCR text gave the same body start and the same label.

3.3 Outcome labels and their validation

Four classes. We predict four mutually exclusive outcomes: *allowed*, *dismissed*, *disposed* (disposed of, typically with directions or as infructuous) and *partly allowed*.

Text-derived label. The label is the latest explicit disposition phrase in the final 20% of the judgment text. Phrases such as "the appeal is accordingly dismissed" or "the appeals are partly allowed" are matched by a fixed set of patterns.

Official label. For archive judgments we also map the official disposal nature onto the same four classes (Table 1). Other values, such as adjourned or referred, have no four-way equivalent.

Agreement and filtering. On the 341 SC-2016 training judgments where both labels exist, they agree on 333 (97.7%). An archive judgment is retained only when both labels exist and agree. This filter costs some coverage but removes most label noise from the added data.

Table 1. Mapping from the archive's official disposal nature to the four outcome classes, and the selection funnel for archive judgments decided in 2013–2015 (training years) and 2017 (test year).

Official disposal nature	Class
Appeal(s) allowed; Case allowed	allowed
Dismissed	dismissed
Disposed off	disposed
Case partly allowed	partly allowed
Any other value	excluded

Selection step (3,142 archive judgments)	Judgments
No explicit disposition phrase in the final 20% of the text	211
Official disposal outside the four classes	162
Text label disagrees with official label	44
Duplicate report; already in SC-2016; decided in another year	4; 2; 3
Retained: training years (2013–2015) / test year (2017)	2,148 / 568

3.4 Splits

Split rules.

- **SC-2016** judgments keep their hash split.
- **Archive training data.** Archive judgments decided in 2013–2015 join the training split.
- **Archive test data.** Archive judgments decided in 2017 form an out-of-time test set, decided after every archive training judgment.
- **2016 archive judgments** are excluded, so that no 2016 judgment can enter the data twice through the two sources.
- **De-duplication.** Duplicates are removed by report identifier, both within the archive and against SC-2016.

Final counts. An example also needs a valid input view (Section 3.5). After that filter, the benchmark has 2,470 training examples (731, 699 and 683 archive judgments from 2013, 2014 and 2015, plus 357 from SC-2016), 46 validation examples (SC-2016) and 608 test examples (50 from SC-2016 and 558 from 2017). Figure 2 shows the composition. Labels are imbalanced: *allowed* is the largest class and *partly allowed* the smallest, at 134 of 2,470 training examples.

Date anomalies. Three SC-2016 judgments carry implausible OCR-parsed decision years. They are kept in their hash-assigned splits, and their earlier dates only make graph retrieval more conservative.

Leakage checks. We verified that:

- the SC-2016 training, validation and test examples are byte-identical to those of the earlier benchmark version;
- no 2017 judgment contributes a training-split graph edge.

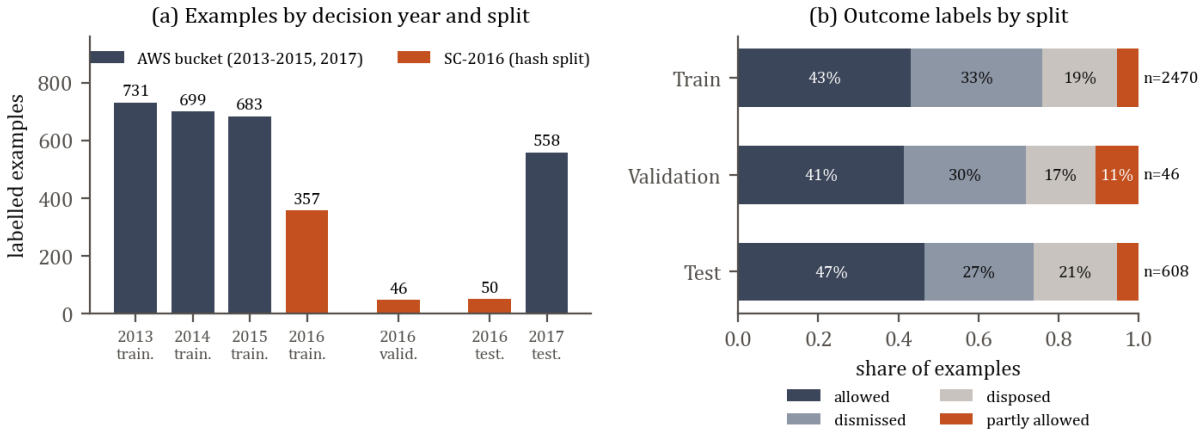


Figure 2. Benchmark composition. (a) Labelled examples by decision year, source and split; the 2017 test year lies after all training years. (b) Share of each outcome label per split; *n* is the number of examples.

3.5 A masked, leakage-controlled input view

Why a masked view. Judgments announce their own outcome, so the input must exclude the passages that state it. An earlier version of our input view began at the top of the document and exposed the SCR headnote. Headnotes summarise the holding, for example "Dismissing the appeal, the Court HELD", and so leak the outcome directly.

The final view. The final view (`body-prefix-masked-v2`) works as follows:

- **Start.** It starts at the judgment body: the "judgment of the Court was delivered by" line, or else the jurisdiction block, or else an ORDER/JUDGMENT heading.
- **Length.** It takes at most 6,000 characters and never reads beyond 70% of the document. Labels come from the final 20%, so a gap of at least 10% always separates the view from the label region.
- **Masking.** Every explicit disposition phrasing is replaced by the token `outcome_phrase`. This covers "allowing/dismissing/disposing of the appeal", "the appeal is allowed/dismissed", "we allow/dismiss", "deserves to be allowed/dismissed/set aside/quashed/upheld" and "HELD".

- **Exclusion.** Judgments without a body marker, or with fewer than 500 characters before the cap, are excluded rather than shown a view that might contain the headnote.

The view therefore carries the case's facts, procedural history and the beginning of the arguments, which is the information closest to what is known before decision.

4 Method

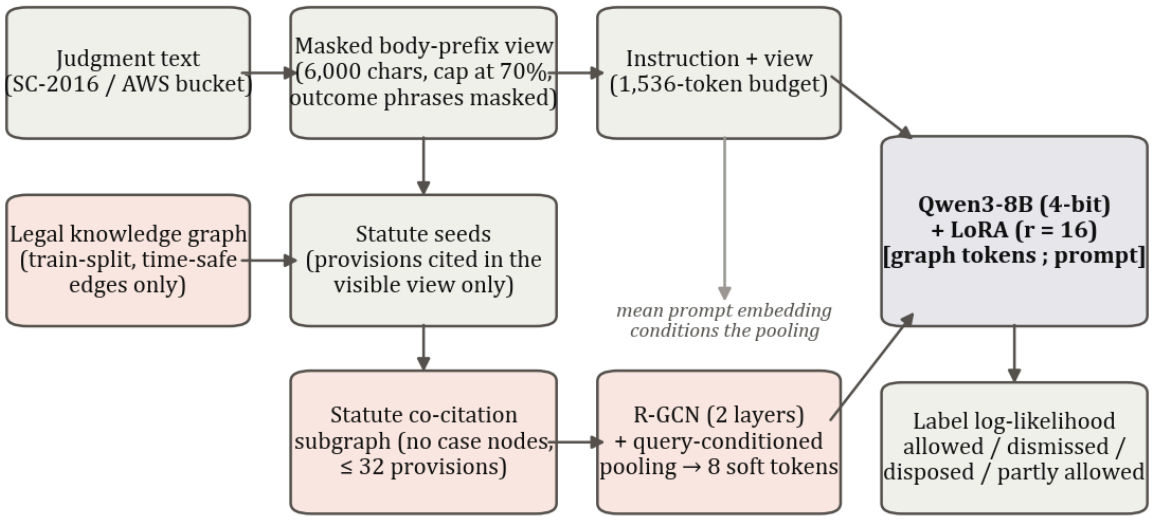


Figure 1. Statute co-citation graph prefix. Provisions cited in the masked view seed a subgraph built from training-split, time-eligible edges only. Case nodes are collapsed into co-citation links between provisions. A two-layer relational GCN and query-conditioned pooling produce eight soft tokens, which are prepended to the instruction and judgment view of a 4-bit Qwen3-8B with LoRA adapters. The outcome is the label whose verbalisation has the highest log-likelihood.

4.1 Task formulation and scoring

Prompt. Let x be the masked view of a judgment and $\mathcal{Y} = \{\text{allowed}, \text{dismissed}, \text{disposed}, \text{partly allowed}\}$. The prompt is a fixed instruction, followed by x (truncated to fit a 1,536-token budget) and an answer cue (Appendix A). Each label y has a short verbalisation $v_y = (v_{y,1}, \ldots, v_{y,T_y})$.

Scoring. Given the prompt embedding sequence $\mathbf{E}(x)$ and an optional graph prefix $\mathbf{P}(x, G)$, the model scores each label by the total log-probability of its verbalisation:

$$s(y \mid x) = \sum_{t=1}^{T_y} \log p_\theta\big(v_{y,t} \mid [\mathbf{P}(x, G);\, \mathbf{E}(x)],\, v_{y,<t}\big), \qquad \hat{y} = \arg\max_{y \in \mathcal{Y}} s(y \mid x).$$

Scoring needs one forward pass over the prompt and short continuations for the multi-token labels. The model therefore always returns one of the four classes and cannot generate free text.

4.2 Legal knowledge graph

Nodes and edges. The graph $G = (V, E)$ is typed and provenance-backed. Nodes are judgments (cases), statute provisions, acts and the court. Edges include:

- `refers_to_provision`: from a judgment to each provision it cites;
- `contains`: from an act to its provisions;
- `decided_by`: from a judgment to the court.

Citation extraction. Citations are extracted from judgment text by a pattern-based extractor. It recognises section and article references with their act ("section 302 of the IPC", "Article 21", "s. 482 CrPC") and resolves each to a provision node.

Provenance. Every edge records the split of the judgment it was extracted from (`origin_split`) and its date.

Size. The graph has 5,931 nodes (3,305 judgments, 2,619 provisions) and 17,076 edges. Training-split judgments contribute 9,173 citation edges, which link 1,705 judgments to 996 distinct provisions. Most of these citations are to the IPC (3,722) and the Constitution (3,406), followed by the CrPC (1,593) and the IEA (452).

4.3 Time-safe and split-safe retrieval

Eligible edges. For a query judgment q with as-of date $\tau(q)$, an edge e is eligible if three conditions hold:

- it was extracted from a training-split judgment;
- its date, and the dates of its endpoints, do not exceed $\tau(q)$;
- neither endpoint is q itself, whose edges were extracted from its full text.

We use the first day of the decision year as $\tau(q)$. This bound is conservative, and it is needed because some SC-2016 judgments lack a full decision date.

Seeds. The seed set S_q contains the provisions resolved from the masked view x, never from the full judgment, that are eligible at $\tau(q)$.

Consequences. Validation and test judgments never contribute edges, and no judgment decided after the query can influence its subgraph. A real user could therefore have computed the same subgraph at decision time.

4.4 Statute co-citation selector

Candidates. Let C_q be the eligible training judgments, and $\mathrm{Cit}(c)$ the provisions judgment c cites through eligible edges. A provision $p \notin S_q$ becomes a candidate when some eligible judgment cites it together with a seed. For each candidate we record the set of seeds it is co-cited with and the number of supporting judgments:

$$A(p) = \bigcup_{c \in C_q \colon p \in \mathrm{Cit}(c)} \big(\mathrm{Cit}(c) \cap S_q\big), \qquad n(p) = \big|\{c \in C_q : p \in \mathrm{Cit}(c),\ \mathrm{Cit}(c) \cap S_q \neq \emptyset\}\big|.$$

Selection. Candidates are ranked by $|A(p)|$, then by $n(p)$ (both descending), then by identifier. The selected node set is the seeds followed by the top-ranked candidates, up to a budget of $B = 32$ nodes.

Edges. Two selected provisions are linked by a `co_occurs_with` edge, in both directions, when an eligible judgment cites both.

Rationale. The construction is the legal analogue of bibliographic co-citation [31]. Provisions that courts repeatedly invoke together, such as a penal section, the procedural provision under which an appeal is brought, and the evidentiary rule on which it turns, describe the legal frame of a dispute.

No case nodes. Judgment nodes are collapsed away. No case title, party name or outcome can enter the prefix, and the subgraph has the same meaning at training and test time.

Comparison with PCST. Our earlier selector, the prize-collecting Steiner tree formulation used by G-Retriever [23, 55], produced subgraphs whose nodes were mostly co-citing judgments (Section 6.5).

4.5 Graph encoder and soft-token prefix

Node features. Each node v has a feature vector $\mathbf{x}_v \in \mathbb{R}^d$ ($d = 4096$). It is the mean of the frozen LLM input embeddings over the first 48 tokens of the node's text: the provision's label and the first 400 characters of its text. The node encoder therefore shares the LLM's vocabulary geometry without running the LLM over the graph.

Message passing. After an input projection to $h = 256$ dimensions, two relational GCN layers [32] update each node:

$$\mathbf{h}_v^{(l+1)} = \mathrm{ReLU}\Big(\mathbf{W}_0^{(l)}\mathbf{h}_v^{(l)} + \sum_{r\in\mathcal{R}} \sum_{u\in\mathcal{N}_r(v)} \tfrac{1}{c_{v,r}}\,\mathbf{W}_r^{(l)}\mathbf{h}_u^{(l)}\Big),$$

where $\mathcal{N}_r(v)$ is the set of neighbours of v under relation r, $c_{v,r} = |\mathcal{N}_r(v)|$, and $\mathcal{R}$ includes inverse relations.

Pooling. The node states $\mathbf{H} \in \mathbb{R}^{|V_q|\times h}$ are pooled into $K = 8$ tokens by attention with learned queries $\mathbf{Q}_0 \in \mathbb{R}^{K\times h}$. The queries are shifted by a projection of the mean prompt embedding $\bar{\mathbf{e}}$, which makes the pooling query-conditioned:

$$\mathbf{Q} = \mathbf{Q}_0 + \mathbf{1}\,(\mathbf{W}_q\bar{\mathbf{e}})^\top, \qquad \boldsymbol{\alpha} = \mathrm{softmax}\Big(\tfrac{\mathbf{Q}(\mathbf{W}_k\mathbf{H}^\top)^\top}{\sqrt{h}}\Big), \qquad \mathbf{P} = \gamma\cdot \mathrm{LN}\big(\mathrm{MLP}(\boldsymbol{\alpha}\mathbf{H})\big).$$

Output scale and empty subgraphs. A two-layer MLP maps the pooled states to the LLM's d-dimensional embedding space. The learned scale γ is initialised to the root-mean-square of the LLM's input embeddings, so the prefix starts at the scale of ordinary tokens. When a query has no eligible seed, its subgraph is empty and the model receives K learned *null tokens*. The graph model therefore receives the same eight prefix positions for every case. The encoder has 6,340,097 parameters.

4.6 Conditioning and training objective

Base model and adapters. The prefix is concatenated before the prompt embeddings, $[\mathbf{P}; \mathbf{E}(x)]$. The base model is Qwen3-8B [59], loaded in 4-bit NormalFloat with double quantisation and float16 compute [14]. LoRA adapters of rank 16 ($\alpha = 32$, dropout 0.05) are attached to all attention and MLP projections [13], which gives 43,646,976 trainable adapter parameters.

Objective. Training minimises a class-weighted cross-entropy over the verbalised label and an end-of-sequence token:

$$\mathcal{L}(x, y) = -\, w_y \sum_{t=1}^{T_y+1} \log p_\theta\big(v_{y,t} \mid [\mathbf{P}; \mathbf{E}(x)],\, v_{y,<t}\big), \qquad w_y \propto \frac{N}{|\mathcal{Y}|\, N_y},$$

where the weights are inverse class frequencies normalised to a mean of one over the training examples [60]. The resulting weights are 0.58 for *allowed*, 0.76 for *dismissed*, 1.34 for *disposed* and 4.61 for *partly allowed*.

Optimisation. Adapters and encoder are trained jointly with AdamW, using learning rates of 2×10^{-4} (LoRA) and 5×10^{-4} (encoder). The schedule has 10% linear warm-up followed by linear decay. We accumulate gradients over 8 examples and clip gradient norms at 1.0. An update is skipped, and counted, if its loss or gradient is non-finite; no such event occurred in the reported runs.

4.7 Baselines and controls

Baselines. All LLM baselines use the same base model, prompt, input view and scoring rule.

- **Majority** always predicts *allowed*.
- **Zero-shot** scores the four labels with the untuned base model.
- **BM25 evidence prompt** retrieves the four statute provisions most relevant to the view with BM25 [63]. It prepends up to 384 tokens of their text to the prompt, inside the same 1,536-token budget, without training.
- **QLoRA text** is identical to the graph model except that it has no graph encoder. It uses the same data, seed, data order, recipe and budget.

Control. Finally, the trained graph model is scored a second time with its graph tokens replaced by the null tokens (*graph-token ablation*). This isolates the per-case contribution of the subgraph at inference time.

5 Experimental Setup

5.1 Compute and software

Hardware and software. All runs used a single NVIDIA T4 GPU (16 GB) in a hosted notebook service with a weekly GPU quota, with PyTorch 2.11.0+cu128, Hugging Face Transformers [64], PEFT and bitsandbytes. The base model is pinned to a fixed repository revision. Each notebook embeds the exact project code and configuration as a deterministic archive and verifies the SHA-256 of every input before training.

Run time. One epoch over 2,470 examples took 7.38 h for the text model and 7.43 h for the graph model (10.8 and 10.8 s per example). The graph prefix therefore costs under one percent of training time.

Checkpoints. Training writes resumable checkpoints every 96 examples: adapter, encoder, optimiser, scheduler, random states and data order. 25 checkpoints were written per run.

Table 2. Training configuration shared by the text and graph models. The two configurations differ only in the variant and the selector.

Setting	Value
Base model	Qwen3-8B, 4-bit NF4, double quantisation, fp16 compute
LoRA	rank 16, α = 32, dropout 0.05; q, k, v, o, gate, up, down projections
Learning rate (LoRA / encoder)	2×10^{-4} / 5×10^{-4}, AdamW
Schedule	10% warm-up, linear decay; 1 epoch; gradient accumulation 8
Loss	inverse-frequency class weights; gradient clipping 1.0
Input	masked body prefix, ≤ 6,000 characters; prompt budget 1,536 tokens
Graph prefix	co-citation selector, budget 32 nodes; R-GCN 2 × 256; 8 tokens; 48-token node text
Seed; data order	20261006; identical for both models
Checkpointing	every 96 examples (resumable)

5.2 Evaluation protocol

Training and freezing. Each model is trained in its own session. The session never reads test labels; this is recorded as `test_labels_accessed: false` in the run manifest. At the end, the run is *frozen*: a manifest records the SHA-256 of the adapter files, the encoder weights and the configuration.

Single test pass. A separate evaluation session takes the frozen run as input and verifies every hash. It then scores the test split exactly once and refuses to run again if a result exists. Graph models are scored once more with graph tokens ablated. Test predictions from both test sources are produced in this single pass and reported separately.

Validation. The validation set was used to monitor training and to choose among development designs (Section 6.5), never for test-time decisions.

5.3 Metrics and statistics

Metrics. Macro-averaged F1 over the four classes is the primary metric, because the classes are imbalanced. We also report accuracy and per-class F1.

Intervals and tests.

- **Confidence intervals** are 95% percentile bootstrap intervals over cases, with 10,000 resamples [34].
- **Paired comparisons** resample cases jointly for both systems and report the 95% interval of the difference [28]. Accuracy differences are also tested with the exact McNemar test on discordant cases [35].

6 Results

6.1 Main results on the held-out test sets

Table 3 and Figure 5 report every system on the two test sources and on their union.

Majority baseline. Because *allowed* is the most frequent class, the majority baseline reaches an accuracy of 0.465 but a macro-F1 of only 0.159.

Fine-tuned models. Both fine-tuned models more than double the majority macro-F1 on the 2017 set and on the full test set: the text model reaches 0.360 and the graph model 0.365 on all 608 cases.

Untrained baselines. Fine-tuning is clearly beneficial. Used without training, the same model reaches a macro-F1 of 0.269. Its predictions concentrate on *allowed*, *dismissed* and *partly allowed*, and it rarely predicts *disposed* (30 of 608 predictions). Prepending BM25-retrieved statute text raises accuracy to 0.438, but only by pushing predictions towards *allowed* (425 of 608), and macro-F1 falls to 0.255. Both fine-tuned models improve on both untrained baselines by about 0.09–0.11 macro-F1, and every such interval excludes zero (Table 4). Over the zero-shot model, the gain is +0.090 for the text model and +0.096 for the graph model. Over the BM25 prompt, it is +0.104 and +0.110.

2017 out-of-time test. The 2017 set has 558 judgments, all decided after every training judgment. On it, the statute-graph model attains the highest macro-F1 of all systems, 0.379 (95% CI 0.330–0.430), against 0.365 (0.322–0.409) for the text-only model. The text-only model has the higher accuracy: 0.489 against 0.462.

SC-2016 test. On the 50-judgment SC-2016 test set the text-only model leads on both metrics. Its macro-F1 is 0.316 against 0.226, and its accuracy 0.440 against 0.320. Intervals on this set are more than twice as wide as on the 2017 set.

Table 3. Test results from a single evaluation of frozen models: accuracy (Acc) and macro-F1 (mF1). Brackets give 95% bootstrap confidence intervals for macro-F1. Bold marks the best model row in each column; the majority baseline is excluded, and it has the highest SC-2016 accuracy. The last row scores the trained graph model with its graph tokens replaced by learned null tokens. All LLM rows use Qwen3-8B with the same prompt and masked input view.

System	SC-2016 Acc	SC-2016 mF1	2017 Acc	2017 mF1 [95% CI]	All Acc	All mF1 [95% CI]
Majority (always allowed)	0.500	0.167	0.462	0.158	0.465	0.159
Zero-shot	0.340	0.250	0.367	0.268 [0.232, 0.305]	0.365	0.269 [0.233, 0.303]
BM25 statute-evidence prompt	0.320	0.211	0.448	0.259 [0.229, 0.289]	0.438	0.255 [0.227, 0.284]
QLoRA text	**0.440**	**0.316**	**0.489**	0.365 [0.322, 0.409]	**0.485**	0.360 [0.320, 0.400]
QLoRA + statute graph prefix	0.320	0.226	0.462	**0.379** [0.330, 0.430]	0.451	**0.365** [0.320, 0.412]
graph tokens removed	0.320	0.223	0.461	0.378 [0.329, 0.427]	0.449	0.364 [0.319, 0.407]

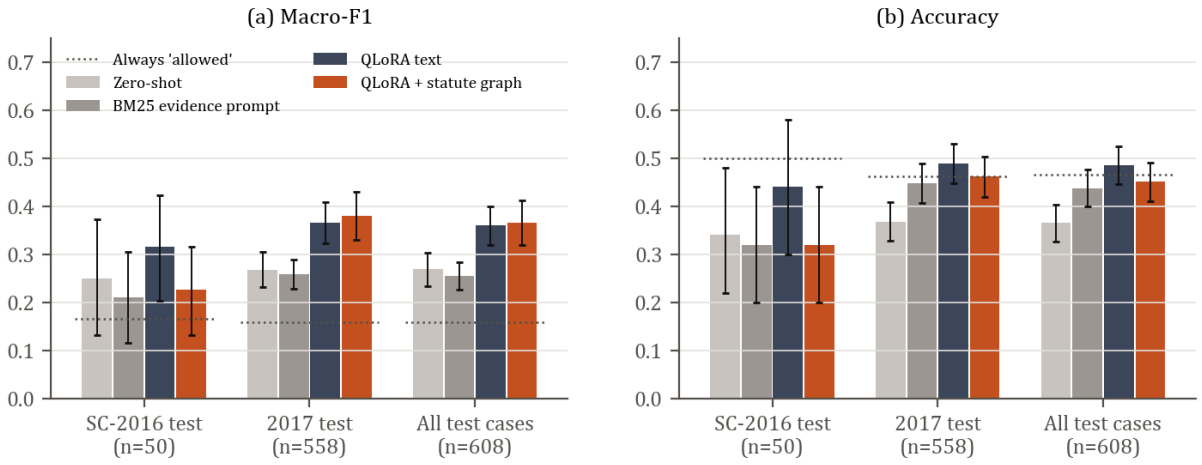


Figure 5. Test macro-F1 (a) and accuracy (b) for each system and test source, with 95% bootstrap confidence intervals. The dotted line marks the majority (always *allowed*) baseline.

6.2 Paired comparisons

Graph vs text. Table 4 compares systems on the same cases.

- **2017 test.** The graph model's macro-F1 advantage is +0.014 (CI −0.028 to +0.058); it is positive in 73% of bootstrap resamples. The accuracy difference (graph − text) of −0.027 is not significant (McNemar $p = 0.20$). Neither model dominates.
- **SC-2016 test.** The text model's lead is larger: the macro-F1 interval excludes zero, while the McNemar test on accuracy gives $p = 0.07$, based on 7 versus 1 discordant cases.
- **Pooled.** Over all 608 cases the two models are level on macro-F1 (+0.006), and the accuracy difference is not significant ($p = 0.08$).

Table 4. Paired differences (first system minus second) on the same test cases. Δ values are point differences; brackets give 95% paired bootstrap intervals; *p* is the exact McNemar test on accuracy.

Comparison	Test set	Δ macro-F1 [95% CI]	Δ accuracy [95% CI]	McNemar *p*
Graph − text	2017 (n = 558)	+0.014 [−0.028, +0.058]	−0.027 [−0.065, +0.011]	0.20
	SC-2016 (n = 50)	−0.090 [−0.179, −0.007]	−0.120 [−0.220, −0.020]	0.07
	All (n = 608)	+0.006 [−0.034, +0.046]	−0.035 [−0.071, +0.002]	0.08
Text − zero-shot	All	+0.090 [+0.037, +0.143]	+0.120 [+0.069, +0.171]	< 0.001
Graph − zero-shot	All	+0.096 [+0.038, +0.153]	+0.086 [+0.031, +0.140]	0.003
Graph − BM25 prompt	All	+0.110 [+0.056, +0.164]	+0.013 [−0.043, +0.067]	0.677
Graph − graph tokens removed	All	+0.002 [−0.022, +0.021]	+0.002 [−0.013, +0.016]	1.00

6.3 Class-level behaviour

Prediction distribution. The two fine-tuned models distribute their predictions differently (Figure 6). On the 2017 set the gold distribution is 258 allowed, 154 dismissed, 118 disposed and 28 partly allowed. Predictions by class:

- **Text model:** 317 allowed, 74 dismissed, 140 disposed, 27 partly allowed.

- **Graph model:** 232 allowed, 104 dismissed, 210 disposed, 12 partly allowed.

The text model leans towards *allowed*, the majority class. The graph model shifts mass towards *dismissed* and *disposed*.

Per-class F1 on the 2017 set.

- **Dismissed:** F1 rises from 0.272 to 0.395, and recall from 0.20 to 0.33.
- **Partly allowed:** F1 rises from 0.109 to 0.150.
- **Allowed:** F1 falls from 0.630 to 0.551.
- **Disposed:** F1 is similar (0.450 vs 0.421).

This pattern is what lifts macro-F1, which weighs every class equally, while lowering accuracy, which is dominated by the *allowed* class.

Hardest class. Across all systems, *partly allowed* remains the hardest class. It has only 134 training examples, and its boundary with *allowed* often depends on the operative order rather than on the facts.

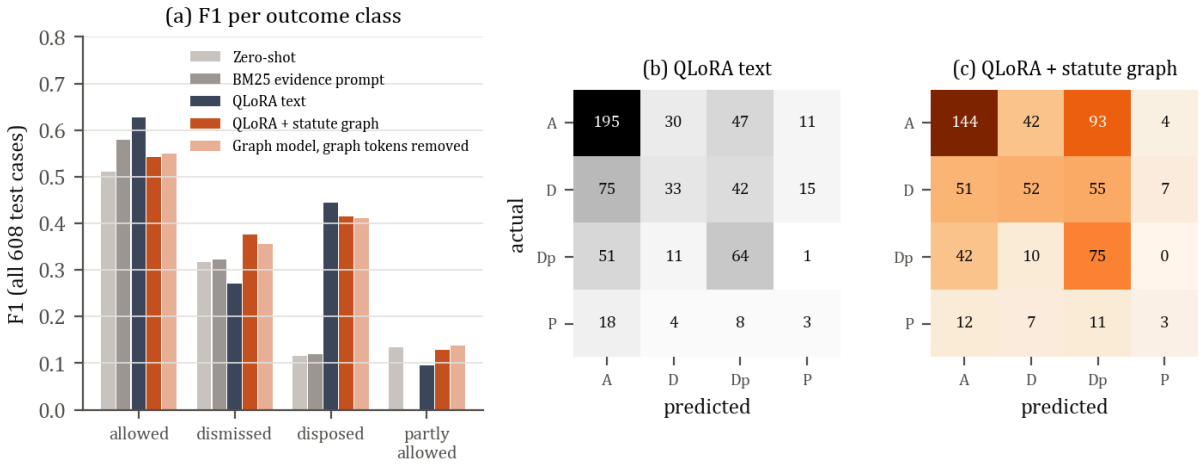


Figure 6. (a) F1 per outcome class on all 608 test cases. (b, c) Confusion matrices of the text and graph models (rows: actual; columns: predicted; A = allowed, D = dismissed, Dp = disposed, P = partly allowed).

6.4 Where the graph prefix acts

Coverage and subgraph size. The prefix is informative only when the masked view cites at least one provision that is eligible as a seed. Figure 3 summarises the coverage:

- **Coverage.** 44.8% of training cases, 39.1% of validation cases and 35.9% of test cases (218 of 608) receive a non-empty subgraph.
- **Size.** A non-empty subgraph grows from about 3.0 seeds to 31.7 provisions on average, close to the budget of 32, with 855 directed co-citation edges.
- **Content.** Every node is a statute provision.

Cases with a subgraph. On the 218 test cases that receive a subgraph (Table 5), the graph model attains the highest macro-F1: 0.403, against 0.381 for the text model and 0.385 for its own ablation.

Class-level gains. On these cases, F1 for *partly allowed* rises from 0.158 (text) to 0.273, and F1 for *dismissed* from 0.242 to 0.339. The subset has 218 cases, so the intervals remain wide: the graph – text difference is +0.022 [−0.048, +0.093].

Cases without a subgraph. Here the graph and ablated models are identical by construction, since both receive the null tokens.

Ablation. Removing the graph tokens at test time changes 32 predictions (5.3%), all of them among cases with a subgraph. So the per-case graph signal acts selectively. Most of the behavioural difference between the graph and text models, including the redistribution across classes in Section 6.3, is carried by the adapter weights that were trained jointly with the prefix rather than by the per-case tokens alone. Section 8 describes the seed-matched controls needed to apportion the effect between the two.

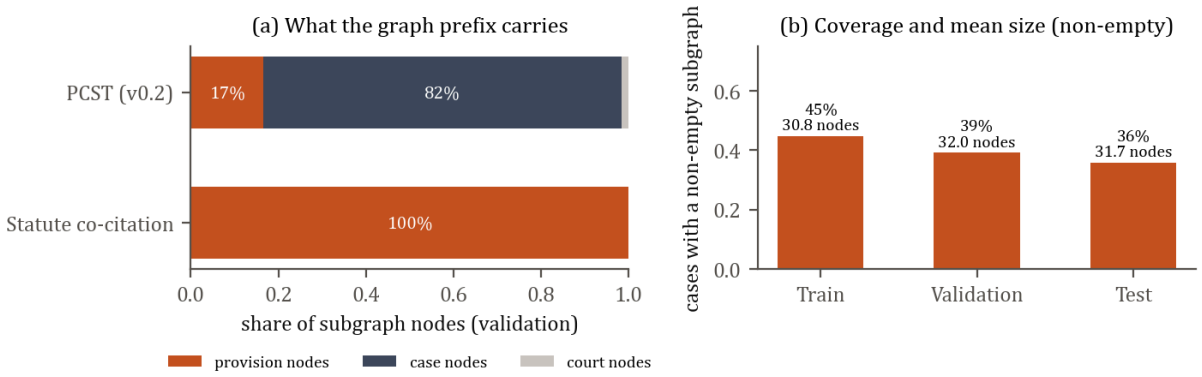


Figure 3. (a) Node composition of query subgraphs on the validation set. The earlier PCST selector on the v0.2 graph filled its budget mostly with co-citing judgments, whose titles were the only text they could contribute; the statute co-citation selector uses provisions only. (b) Share of cases with a non-empty subgraph and the mean subgraph size (non-empty cases) per split.

Table 5. Test results split by whether a case received a non-empty subgraph (accuracy / macro-F1). Without a subgraph, the graph model and its ablation coincide by construction.

Subset	n	QLoRA text	QLoRA + graph	Graph tokens removed
With subgraph	218	0.472 / 0.381	0.450 / **0.403**	0.445 / 0.385
Without subgraph	390	0.492 / 0.334	0.451 / 0.334	0.451 / 0.334

6.5 Development stages

Table 6 and Figure 7 trace the validation results of every 8B development run. All runs use the same 46 validation judgments.

Stage 1. The first stage used the v0.2 benchmark (357 SC-2016 training examples), an unweighted loss and a constant learning rate of 10^{-4}.

- **Text model.** It peaked at a macro-F1 of 0.441 in its second epoch and fell to 0.168 in the third.
- **Graph model.** The PCST graph variant reached 0.244.
- **Saul-7B.** A text-only Saul-7B run under the same recipe stayed close to the majority class (0.183).

Stage 2. The balanced recipe (class weights, warm-up and linear decay, learning rate 2×10^{-4}, gradient clipping) made training stable: macro-F1 improved in every epoch for both models. The graph model rose to 0.283.

Stage 3. The final stage combined the 2013–2015 archive training data with the statute co-citation prefix. The graph model reached 0.385 and the text model 0.405, the closest the two have been. A diagnosis between stages 2 and 3 motivated the statute-only design:

- On the 18 validation cases that received a PCST subgraph, the graph models were right far less often than the matching text models. Pooled over the stage-1 and stage-2 runs, only 2 cases were answered correctly by the graph model alone, against 9 by the text model alone.
- Most PCST nodes were co-citing judgments represented only by their titles (Figure 3a): 251 judgment nodes, against 51 provisions and 5 court nodes.
- With the statute-only prefix, the corresponding counts on the 18 validation cases that receive a statute subgraph were 1 and 2.

Table 6. Development history on the 46-case validation set (accuracy and macro-F1 of the selected epoch). v0.2: 357 SC-2016 training examples; v0.3: 2,470 examples (SC-2016 and archive 2013–2015). Original recipe: unweighted loss, constant learning rate 10^{-4}. Balanced recipe: class-weighted loss, warm-up and linear decay, learning rate 2×10^{-4}, gradient clipping.

Model	Data	Recipe	Epochs	Acc	mF1
Qwen3-8B zero-shot	—	no training	0	0.348	0.318
Qwen3-8B + BM25 statute evidence	—	no training	0	0.391	0.214
Qwen3-8B text	v0.2	original	3	0.565	0.441
Qwen3-8B graph (typed, PCST)	v0.2	original	3	0.435	0.244
Saul-7B text	v0.2	original	3	0.435	0.183
Qwen3-8B text	v0.2	balanced	3	0.478	0.377
Qwen3-8B graph (typed, PCST)	v0.2	balanced	3	0.435	0.283
Qwen3-8B text	v0.3	balanced	1	0.457	0.405
Qwen3-8B graph (statute co-citation)	v0.3	balanced	1	0.413	0.385

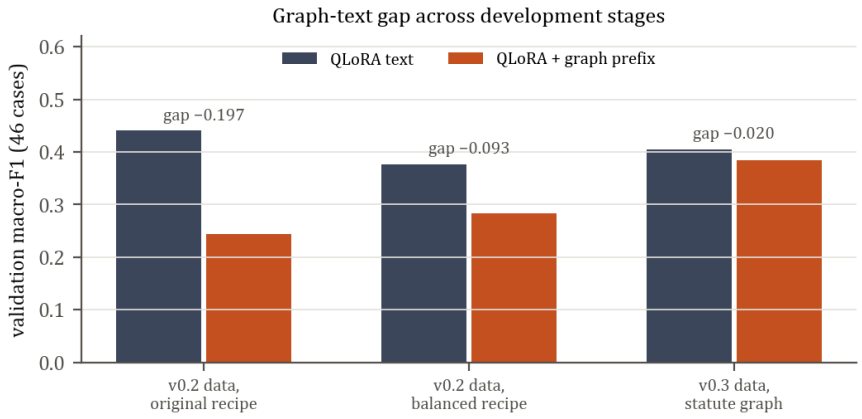


Figure 7. Validation macro-F1 of the text-only and graph-conditioned Qwen3-8B models at the three development stages. The label above each pair gives the graph − text difference.

6.6 Training dynamics and cost

Loss curves. Figure 4 shows the training loss of the final two runs. Both runs see the same examples in the same order, so their curves can be compared step by step.

- **Start of training.** The graph model starts from a much lower loss. Between roughly 200 and 500 examples, the text model's loss is slightly lower.
- **Later.** After about 600 examples the two curves track each other closely. The final ~100-example loss is 0.580 for the text model and 0.590 for the graph model.
- **Stability.** Neither run recorded a non-finite loss or gradient.

Cost. The prefix adds 6,340,097 trainable parameters to the 43,646,976 LoRA parameters. It raises the trainable fraction of the 4-bit model from 0.92% to 1.05%, and increases wall-clock time per example by under one percent.

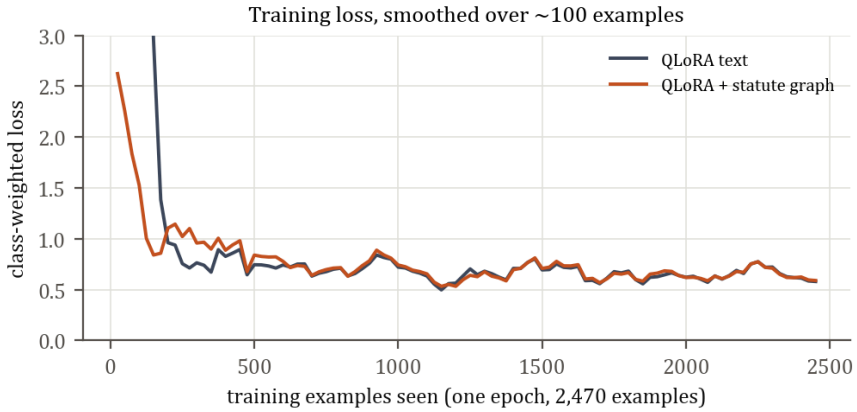


Figure 4. Class-weighted training loss, smoothed over about 100 examples, for the text-only and graph-conditioned models over one epoch of 2,470 examples. The curve is recovered from the running mean logged every 25 examples.

6.7 Diagnostic probes: what limits performance?

Setup. To understand which levers matter most (RQ3), we ran fast probes that use only training-split data. A TF-IDF and logistic-regression classifier was trained on the 1,430 archive judgments from 2013–2014 and scored on the 683 judgments from 2015. This time-ordered development set is about fifteen times larger than the validation set. Table 7 and Figure 8 report the results. Absolute numbers are not comparable with Table 3, because the evaluation years differ; the probes are meant to compare conditions with each other.

Findings.

- **Input view.** Changing the view barely moves the probe. A four-times longer prefix, the last 6,000 characters before the cap and the whole body up to the cap all stay within about 0.03 macro-F1 of the current view. Case details (case type, bench size, party names) add almost nothing.
- **Training-set size.** Macro-F1 still rises with more data: from 0.265 with 178 cases to 0.379 with 1,430.
- **Task framing.** This moves results the most. Merging *partly allowed* into *allowed* raises macro-F1 from 0.379 to 0.442. A binary allowed-versus-dismissed framing, in the style of ILDC [10], reaches 0.604 macro-F1 and 0.633 accuracy.

These findings shape the research directions in Section 8.

Table 7. Diagnostic probes: TF-IDF (1–2-grams) with class-balanced logistic regression. Training uses archive judgments from 2013–2014 (1,430) and scoring uses those from 2015 (683); only training-split data are used. Learning-curve points below the full size average three random subsamples.

Probe	Acc	mF1
Majority (always allowed)	0.425	0.149
Current view: body prefix, 6,000 characters	0.483	0.379
Longer prefix: 24,000 characters	0.493	0.372
Last 6,000 characters before the 70% cap	0.460	0.354
Whole body up to the 70% cap	0.486	0.368
Case details only (type, bench, parties)	0.341	0.276
Current view + case details	0.488	0.381
Training cases: 178 / 357 / 715 / 1,430	0.432 / 0.451 / 0.464 / 0.483	0.265 / 0.291 / 0.338 / 0.379
3-way (partly allowed merged into allowed)	0.515	0.442
Binary: allowed vs dismissed (548 dev cases)	0.633	0.604

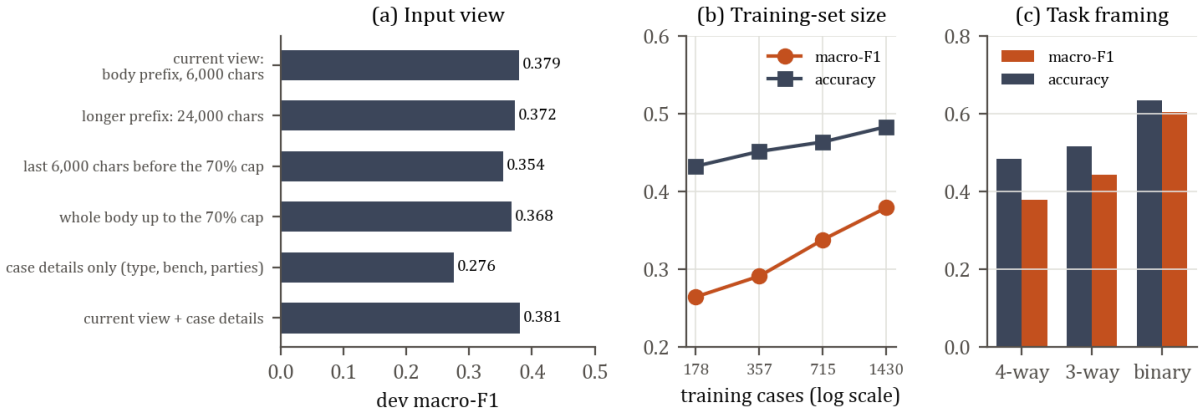


Figure 8. Diagnostic probes on held-out 2015 archive judgments. (a) Input views; (b) learning curve (log scale); (c) task framings.

7 Discussion

7.1 Fine-tuning under strict controls

RQ1. A QLoRA-adapted Qwen3-8B learns a usable outcome signal from the masked opening of a judgment, and it holds up on judgments decided a year after its training data. Three features of the protocol support this reading:

- the input never shows the operative order or headnote;
- the test year lies entirely after the training years;
- each model was scored once, after its weights were frozen and hashed.

Fine-tuning improves macro-F1 by about 0.09–0.11 over the untrained model, whether it is used alone or with retrieved statute text. Unstructured statute evidence in the prompt does not by itself help the untrained model. The resulting numbers are lower than those reported for settings in which the model reads the judgment up to the decision. We regard that as the honest price of a forecasting-style evaluation, in line with recent calls to separate outcome *prediction* from outcome *identification* [24].

7.2 What the statute graph prefix contributes

RQ2. The statute prefix changes what the model predicts, in a direction that matters for balanced performance:

- **2017 test.** On the largest and out-of-time test set it yields the best macro-F1 of all systems.
- **Minority classes.** It recovers clearly more dismissed appeals and more partly-allowed appeals.
- **Cases with a subgraph.** Where a subgraph exists, it improves on both the text model and its own ablation.

These effects fit the legal intuition behind co-citation. A dismissed criminal appeal and an allowed one may share the facts-heavy opening, but differ in the provisions courts habitually invoke alongside the cited sections. Examples are the procedural provisions that govern interference with concurrent findings, or the evidentiary rules on which a conviction rests.

Costs and limits. Two qualifications belong with this reading. First, the text-only model achieves higher accuracy, because it predicts the majority class more often, and it leads on the small SC-2016 test set. On the 2017 set the macro-F1 and accuracy differences between the two fine-tuned models are within paired confidence intervals. Second, the ablation shows that only 5.3% of predictions depend on the per-case tokens at test time. Much of the change in class balance is therefore learned into the adapter during joint training with the prefix. We consider this a property of how soft prefixes are learned, rather than evidence against them: a prefix that shapes the adapter during training can influence predictions even where it is not decisive at inference. A seed-matched control with randomised statute prefixes is the natural next experiment to separate the two mechanisms (Section 8).

Design lesson. The development history yields a practical lesson for graph–LLM fusion in law. Our first design followed the prize-collecting Steiner tree retrieval of G-Retriever [23] and let co-citing judgments enter the subgraph. Because case outcomes may not leak, those judgments could contribute only their titles. The resulting prefix was dominated by names of parties and so carried little legal content. Collapsing judgments into provision–provision co-citation links kept the leakage guarantees, produced compact subgraphs that fill the node budget with statutes, and brought the graph model level with the text model on validation. For legal outcome tasks, *what* the graph carries matters at least as much as *how* it is encoded.

7.3 Out-of-time generalisation and source effects

Comparison of the test sets. The two test sources differ in size, era and text pipeline. The 2017 set is ten times larger than the SC-2016 test set, so its intervals are less than half as wide. Its judgments come from the same PDF text pipeline as 86% of the training data. The SC-2016 test judgments come

from the OCR pipeline used for the remaining 14%. Small differences in text form, together with the small sample, may explain why the two test sets rank the fine-tuned models differently. Per-source reporting, fixed before evaluation, makes such differences visible rather than averaging them away.

Stability over time. The 2017 results of the text model (macro-F1 0.365) are close to its validation result (0.405). This suggests that the model's errors stem from the limited information in the masked opening more than from drift between 2016 and 2017.

7.4 What bounds performance

RQ3. The probes suggest that the masked opening carries a bounded amount of outcome signal. A linear bag-of-words model trained on 2013–2014 reaches a macro-F1 on 2015 judgments in the same range as the 8B models on 2017. Longer views drawn from the same pre-decision part of the judgment add little. Two levers stand out instead:

- **More training data.** The learning curve has not flattened.
- **The task framing.** A binary framing raises the probe's macro-F1 by about 0.23. The four-way task asks a model to separate procedural dispositions (*disposed of*) and partial relief (*partly allowed*) from merits outcomes. That distinction often depends on the operative order, which a forecasting-style view deliberately withholds.

We therefore see the four-way, masked, out-of-time setting as a demanding benchmark rather than a ceiling on what graph-conditioned models can do.

7.5 Limitations

- **Single seed and single epoch.** Each final configuration was trained once, for one epoch, under a weekly GPU quota on a single T4. Seed variance is therefore not yet quantified. The paired intervals capture sampling variation over test cases but not over training runs.
- **Small validation set.** The validation set has 46 judgments and was used for development choices. The diagnostic probes use a larger, time-ordered development set, but only with linear models.
- **Heuristic labels.** Labels come from disposition phrases and, for archive data, from agreement with official metadata. Judgments without an explicit phrase or with other dispositions are excluded, which may make the benchmark easier than the full docket.
- **Graph coverage.** Only 36% of test cases receive a subgraph. This is limited by citation extraction from the masked view and by the six enactments in the statute layer.
- **Text quality.** Archive text comes from PDF text layers. These are cleaner than OCR in some respects and noisier in others, and recover about 88% of statute citations.
- **No human baseline.** We did not measure expert performance on the same masked views.

7.6 Ethical considerations

Data. All judgments are public records released for reuse. Our graph prefix excludes case titles and party names by construction, so the conditioning signal contains statutory text only.

Use. Outcome prediction systems must not be used to decide cases or to advise litigants without professional oversight. Errors are unevenly distributed across outcome classes (Figure 6), and LLMs can produce confident but unfounded legal statements [51]. We present the models as research instruments for studying legal structure and evaluation methodology, not as decision aids.

8 Further Research

1. **Task framings aligned with legal practice.** Report binary (allowed vs dismissed) and three-way framings alongside the four-way task. Treat *disposed of* as a separate procedural prediction, consistent with ILDC [10] and IL-TUR [11]. The probes indicate gains of about 0.23 macro-F1 from the framing alone.
2. **More training years.** The open archive contains labelled judgments from 2010–2012 and earlier years that pass the same official-label check. The learning curve suggests that more data will improve all systems, and that it will enlarge the set of training co-citations available to the graph prefix.
3. **Time-safe precedent signals.** Extend the graph with information about how earlier, similar judgments were decided, using only training-split judgments decided before the query. This is a case-based reasoning signal that complements statutes [50]. Other candidates are typed links to the lower court, bench composition and subject matter.
4. **Higher graph coverage.** Improve citation extraction (abbreviations, state enactments, special statutes) and add further enactments, so that more than 36% of cases receive a subgraph.
5. **Disentangling the prefix and the adapter.** Train multiple seeds of the text and graph models with matched data order, and add a control whose prefix encodes randomly drawn provisions of the same size. This would separate the effect of statute content from the regularising effect of joint training with a prefix.
6. **Structure-aware inputs with audited leakage.** Use rhetorical-role segmentation [41] to select facts and arguments from anywhere before the decision, under explicit leakage audits. Also consider hierarchical encoders that read beyond the 1,536-token budget.
7. **Domain adaptation and legal LLMs.** Continue pre-training on the archive's unlabelled judgments from training years [46]. Compare legal LLMs such as SaulLM [47] under the same protocol.
8. **Explanations grounded in statutes.** Attribute each prediction to provisions in its subgraph. Evaluate these attributions against expert explanations [10, 43] and rationale-annotated data [40].

9 Conclusion

Summary. We introduced a leakage-controlled, out-of-time benchmark for four-way outcome prediction at the Supreme Court of India, together with a statute co-citation graph prefix for quantised LLMs. The prefix is built only from earlier training judgments, carries no case identities or outcomes, and costs under one percent of training time.

Results. Under a single-shot, hash-verified evaluation, both fine-tuned Qwen3-8B models more than doubled the majority-class macro-F1 and significantly improved on the same model used without training. On the 2017 out-of-time test set, the statute-graph model achieved the best macro-F1 of all systems, with markedly better recognition of dismissed and partly allowed appeals. The text-only model achieved higher accuracy.

Development and outlook. The development history shows that graph content is decisive: replacing co-citing case titles with statute co-citation closed most of the validation gap between graph-conditioned and text-only models. Diagnostic probes point to task framing, more training years and time-safe precedent signals as the most promising next steps. Our code, configurations and hashes are released to make each of these steps directly comparable with the results reported here.

Data and Code Availability

Sources. Both judgment sources and the statute sources are public [29, 30, 62].

Repository. The project repository contains the code that downloads, extracts, cleans, labels and splits the data, builds the graph and runs every experiment. It also contains the source registry with archive hashes and the configurations of all runs.

Run artefacts. The run manifests record the SHA-256 of the corpus (`d92f9dd3…`), training manifest (`7b5c2c2f…`) and graph (`c5a733cd…`), and the base-model revision (`b968826d…`).

Paper numbers. All results in Section 6 and all figures are generated by scripts from the frozen runs' outputs.

Acknowledgements

We thank the maintainers of the open judgment archive and of the SC-2016 collection for making the data available, and the providers of hosted GPU notebooks used for all experiments.

Appendix A Prompt and label verbalisers

```
Read the opening of an Indian Supreme Court judgment. Phrases that state the final disposition have
been replaced by outcome_phrase. Answer with the final outcome: allowed, dismissed, disposed, or
partly allowed.

Judgment:
<masked judgment view, truncated to the token budget>

Outcome:
```

Label verbalisers: " allowed", " dismissed", " disposed", " partly allowed". The BM25 baseline prepends up to 384 tokens of retrieved provision text before the instruction. The graph model prepends eight soft tokens before the instruction.

References

[1] N. Aletras, D. Tsarapatsanis, D. Preoţiuc-Pietro, and V. Lampos, “Predicting judicial decisions of the European Court of Human Rights: A natural language processing perspective,” *PeerJ Computer Science*, vol. 2, e93, 2016.

[2] M. Medvedeva, M. Vols, and M. Wieling, “Using machine learning to predict decisions of the European Court of Human Rights,” *Artificial Intelligence and Law*, vol. 28, pp. 237–266, 2020.

[3] I. Chalkidis, I. Androutsopoulos, and N. Aletras, “Neural legal judgment prediction in English,” in *Proc. 57th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2019, pp. 4317–4323.

[4] D. M. Katz, M. J. Bommarito II, and J. Blackman, “A general approach for predicting the behavior of the Supreme Court of the United States,” *PLOS ONE*, vol. 12, no. 4, e0174698, 2017.

[5] H. Zhong, Z. Guo, C. Tu, C. Xiao, Z. Liu, and M. Sun, "Legal judgment prediction via topological learning," in *Proc. Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2018, pp. 3540–3549.

[6] C. Xiao, H. Zhong, Z. Guo, C. Tu, Z. Liu, M. Sun, *et al.*, "CAIL2018: A large-scale legal dataset for judgment prediction," arXiv:1807.02478, 2018.

[7] J. Niklaus, I. Chalkidis, and M. Stürmer, "Swiss-Judgment-Prediction: A multilingual legal judgment prediction benchmark," in *Proc. Natural Legal Language Processing Workshop (NLLP)*, 2021.

[8] H. Zhong, C. Xiao, C. Tu, T. Zhang, Z. Liu, and M. Sun, "How does NLP benefit legal system: A summary of legal artificial intelligence," in *Proc. 58th Annual Meeting of the ACL*, 2020, pp. 5218–5230.

[9] Y. Feng, C. Li, and V. Ng, "Legal judgment prediction: A survey of the state of the art," in *Proc. 31st International Joint Conference on Artificial Intelligence (IJCAI), Survey Track*, 2022, pp. 5461–5469.

[10] V. Malik, R. Sanjay, S. K. Nigam, K. Ghosh, S. K. Guha, A. Bhattacharya, and A. Modi, "ILDC for CJPE: Indian legal documents corpus for court judgment prediction and explanation," in *Proc. ACL-IJCNLP*, 2021, pp. 4046–4062.

[11] A. Joshi, S. Paul, A. Sharma, P. Goyal, S. Ghosh, and A. Modi, "IL-TUR: Benchmark for Indian legal text understanding and reasoning," in *Proc. 62nd Annual Meeting of the ACL*, 2024.

[12] S. Paul, A. Mandal, P. Goyal, and S. Ghosh, "Pre-trained language models for the legal domain: A case study on Indian law," in *Proc. 19th International Conference on Artificial Intelligence and Law (ICAIL)*, 2023.

[13] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "LoRA: Low-rank adaptation of large language models," in *Proc. International Conference on Learning Representations (ICLR)*, 2022.

[14] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, "QLoRA: Efficient finetuning of quantized LLMs," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[15] N. Xu, P. Wang, L. Chen, L. Pan, X. Wang, and J. Zhao, "Distinguish confusing law articles for legal judgment prediction," in *Proc. 58th Annual Meeting of the ACL*, 2020, pp. 3086–3095.

[16] S. Paul, P. Goyal, and S. Ghosh, "LeSICiN: A heterogeneous graph-based approach for automatic legal statute identification from Indian legal documents," in *Proc. AAAI Conference on Artificial Intelligence*, vol. 36, 2022.

[17] P. Bhattacharya, K. Ghosh, A. Pal, and S. Ghosh, "Hier-SPCNet: A legal statute hierarchy-based heterogeneous network for computing legal case document similarity," in *Proc. 43rd International ACM SIGIR Conference*, 2020.

[18] Q. Dong and S. Niu, "Legal judgment prediction via relational learning," in *Proc. 44th International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2021.

[19] M. Yasunaga, H. Ren, A. Bosselut, P. Liang, and J. Leskovec, "QA-GNN: Reasoning with language models and knowledge graphs for question answering," in *Proc. NAACL-HLT*, 2021, pp. 535–546.

[20] X. Zhang, A. Bosselut, M. Yasunaga, H. Ren, P. Liang, C. D. Manning, and J. Leskovec, "GreaseLM: Graph reasoning enhanced language models for question answering," in *Proc. International Conference on Learning Representations (ICLR)*, 2022.

[21] B. Perozzi, B. Fatemi, D. Zelle, A. Tsitsulin, M. Kazemi, R. Al-Rfou, and J. Halcrow, "Let your graph do the talking: Encoding structured data for LLMs," arXiv:2402.05862, 2024.

[22] Y. Tian, H. Song, Z. Wang, H. Wang, Z. Hu, F. Wang, N. V. Chawla, and P. Xu, "Graph neural prompting with large language models," in *Proc. AAAI Conference on Artificial Intelligence*, vol. 38, 2024.

[23] X. He, Y. Tian, Y. Sun, N. V. Chawla, T. Laurent, Y. LeCun, X. Bresson, and B. Hooi, "G-Retriever: Retrieval-augmented generation for textual graph understanding and question answering," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

[24] M. Medvedeva and P. McBride, "Legal judgment prediction: If you are going to do it, do it right," in *Proc. Natural Legal Language Processing Workshop (NLLP)*, 2023.

[25] A. Søgaard, S. Ebert, J. Bastings, and K. Filippova, "We need to talk about random splits," in *Proc. 16th Conference of the European Chapter of the ACL (EACL)*, 2021, pp. 1823–1832.

[26] A. Lazaridou *et al.*, "Mind the gap: Assessing temporal generalization in neural language models," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2021.

[27] S. Kapoor and A. Narayanan, "Leakage and the reproducibility crisis in machine-learning-based science," *Patterns*, vol. 4, no. 9, 100804, 2023.

[28] R. Dror, G. Baumer, S. Shlomov, and R. Reichart, "The hitchhiker's guide to testing statistical significance in natural language processing," in *Proc. 56th Annual Meeting of the ACL*, 2018, pp. 1383–1392.

[29] "Indian-law-supreme-court-judgements-2016," Hugging Face dataset `Shreyasrao/Indian-law-supreme-court-judgements-2016`, accessed October 2026.

[30] "Indian Supreme Court Judgments," Registry of Open Data on AWS, `s3://indian-supreme-court-judgments`, licence CC BY 4.0, accessed 11 October 2026.

[31] H. Small, "Co-citation in the scientific literature: A new measure of the relationship between two documents," *Journal of the American Society for Information Science*, vol. 24, no. 4, pp. 265–269, 1973.

[32] M. Schlichtkrull, T. N. Kipf, P. Bloem, R. van den Berg, I. Titov, and M. Welling, "Modeling relational data with graph convolutional networks," in *Proc. Extended Semantic Web Conference (ESWC)*, 2018, pp. 593–607.

[33] X. L. Li and P. Liang, "Prefix-tuning: Optimizing continuous prompts for generation," in *Proc. ACL-IJCNLP*, 2021, pp. 4582–4597.

[34] B. Efron and R. J. Tibshirani, *An Introduction to the Bootstrap*. New York: Chapman & Hall/CRC, 1993.

[35] Q. McNemar, "Note on the sampling error of the difference between correlated proportions or percentages," *Psychometrika*, vol. 12, no. 2, pp. 153–157, 1947.

[36] L. Yue, Q. Liu, B. Jin, H. Wu, K. Zhang, Y. An, M. Cheng, B. Yin, and D. Wu, "NeurJudge: A circumstance-aware neural framework for legal judgment prediction," in *Proc. 44th International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2021.

[37] Y. Feng, C. Li, and V. Ng, "Legal judgment prediction via event extraction with constraints," in *Proc. 60th Annual Meeting of the ACL*, 2022, pp. 648–664.

[38] I. Chalkidis, A. Jana, D. Hartung, M. Bommarito, I. Androutsopoulos, D. M. Katz, and N. Aletras, "LexGLUE: A benchmark dataset for legal language understanding in English," in *Proc. 60th Annual Meeting of the ACL*, 2022.

[39] J. Niklaus, V. Matoshi, P. Rani, A. Galassi, M. Stürmer, and I. Chalkidis, "LEXTREME: A multi-lingual and multi-task benchmark for the legal domain," in *Findings of the Association for Computational Linguistics: EMNLP 2023*, 2023.

[40] I. Chalkidis, M. Fergadiotis, D. Tsarapatsanis, N. Aletras, I. Androutsopoulos, and P. Malakasiotis, "Paragraph-level rationale extraction through regularization: A case study on European Court of Human Rights cases," in *Proc. NAACL-HLT*, 2021, pp. 226–241.

[41] P. Kalamkar, A. Tiwari, A. Agarwal, S. Karn, S. Gupta, V. Raghavan, and A. Modi, "Corpus for automatic structuring of legal documents," in *Proc. 13th Language Resources and Evaluation Conference (LREC)*, 2022.

[42] S. Vats, A. Zope, S. De, A. Sharma, U. Bhattacharya, S. K. Nigam, S. Guha, K. Rudra, and K. Ghosh, "LLMs – the good, the bad or the indispensable?: A use case on legal statute prediction and legal judgment prediction on Indian court cases," in *Findings of the Association for Computational Linguistics: EMNLP 2023*, 2023.

[43] S. K. Nigam, A. Sharma, D. Khanna, N. Shallum, K. Ghosh, and A. Bhattacharya, "Legal judgment reimagined: PredEx and the rise of intelligent AI interpretation in Indian courts," in *Findings of the Association for Computational Linguistics: ACL 2024*, 2024.

[44] I. Chalkidis, M. Fergadiotis, P. Malakasiotis, N. Aletras, and I. Androutsopoulos, "LEGAL-BERT: The Muppets straight out of law school," in *Findings of the Association for Computational Linguistics: EMNLP 2020*, 2020, pp. 2898–2904.

[45] P. Henderson, M. S. Krass, L. Zheng, N. Guha, C. D. Manning, D. Jurafsky, and D. E. Ho, "Pile of Law: Learning responsible data filtering from the law and a 256GB open-source legal dataset," in *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2022.

[46] S. Gururangan, A. Marasović, S. Swayamdipta, K. Lo, I. Beltagy, D. Downey, and N. A. Smith, "Don't stop pretraining: Adapt language models to domains and tasks," in *Proc. 58th Annual Meeting of the ACL*, 2020, pp. 8342–8360.

[47] P. Colombo, T. P. Pires, M. Boudiaf, D. Culver, R. Melo, C. Corro, A. F. T. Martins, F. Esposito, V. L. Raposo, S. Morgado, and M. Desa, "SaulLM-7B: A pioneering large language model for law," arXiv:2403.03883, 2024.

[48] N. Guha, J. Nyarko, D. E. Ho, C. Ré, *et al.*, "LegalBench: A collaboratively built benchmark for measuring legal reasoning in large language models," in *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2023.

[49] R. Shui, Y. Cao, X. Wang, and T.-S. Chua, "A comprehensive evaluation of large language models on legal judgment prediction," in *Findings of the Association for Computational Linguistics: EMNLP 2023*, 2023.

[50] Y. Wu, S. Zhou, Y. Liu, W. Lu, X. Liu, Y. Zhang, C. Sun, F. Wu, and K. Kuang, "Precedent-enhanced legal judgment prediction with LLM and domain-model collaboration," in *Proc. Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2023.

[51] M. Dahl, V. Magesh, M. Suzgun, and D. E. Ho, "Large legal fictions: Profiling legal hallucinations in large language models," *Journal of Legal Analysis*, vol. 16, no. 1, 2024.

[52] J. H. Fowler, T. R. Johnson, J. F. Spriggs II, S. Jeon, and P. J. Wahlbeck, "Network analysis and the law: Measuring the legal importance of precedents at the U.S. Supreme Court," *Political Analysis*, vol. 15, no. 3, pp. 324–346, 2007.

[53] A. Hogan, E. Blomqvist, M. Cochez, C. d'Amato, G. de Melo, C. Gutierrez, *et al.*, "Knowledge graphs," *ACM Computing Surveys*, vol. 54, no. 4, pp. 1–37, 2021.

[54] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," in *Proc. International Conference on Learning Representations (ICLR)*, 2017.

[55] M. X. Goemans and D. P. Williamson, "A general approximation technique for constrained forest problems," *SIAM Journal on Computing*, vol. 24, no. 2, pp. 296–317, 1995.

[56] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[57] D. Edge, H. Trinh, N. Cheng, J. Bradley, A. Chao, A. Mody, S. Truitt, and J. Larson, "From local to global: A graph RAG approach to query-focused summarization," arXiv:2404.16130, 2024.

[58] S. Pan, L. Luo, Y. Wang, C. Chen, J. Wang, and X. Wu, "Unifying large language models and knowledge graphs: A roadmap," *IEEE Transactions on Knowledge and Data Engineering*, vol. 36, no. 7, 2024.

[59] A. Yang *et al.* (Qwen Team), "Qwen3 technical report," arXiv:2505.09388, 2025.

[60] Y. Cui, M. Jia, T.-Y. Lin, Y. Song, and S. Belongie, "Class-balanced loss based on effective number of samples," in *Proc. IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019, pp. 9268–9277.

[61] Y. Du *et al.*, "PP-OCR: A practical ultra lightweight OCR system," arXiv:2009.09941, 2020.

[62] CivicTech India, Indian law JSON datasets (Indian Penal Code, Code of Criminal Procedure, Indian Evidence Act, Constitution of India), GitHub repositories, commit-pinned, accessed October 2026.

[63] S. Robertson and H. Zaragoza, "The probabilistic relevance framework: BM25 and beyond," *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, pp. 333–389, 2009.

[64] T. Wolf, L. Debut, V. Sanh, J. Chaumond, C. Delangue, A. Moi, P. Cistac, T. Rault, R. Louf, M. Funtowicz, *et al.*, "Transformers: State-of-the-art natural language processing," in *Proc. EMNLP: System Demonstrations*, 2020, pp. 38–45.